Republic of the Philippines
COMISSION ON HIGHER EDUCATION
BAAO COMMUNITY COLLEGE
San Juan, Baao, Camarines Sur

COLLEGE *of* INFORMATION TECHNOLOGY

DOCUMENTATION FOR EMPLOYEE MANAGEMENT DATABASE

In Partial Fulfillment of the Requirements
For the subject in CC 105 Information Management

GROUP 5 THE CODER

Bedural, Marian M.

Maniscan, Benedict B.

Benitez, Dominic G.

Ilano, Angelica B.

Ali, Riza Joy

Bustarga, Ronald

Cabaltea, Jenny

Oliva, Ace Xander

OCTOBER 2026

INTRODUCTION

The **Employee Management Database** is a sample database created using MySQL to organize and manage employee information. The SQL file named employee management.sql contains mock data designed for academic purposes, database testing, and SQL query practice. The data represents a company's employee records, departments, job positions, and salary information.

The database consists of four related tables: departments, positions, employees, and employee_salaries. It contains 10 departments, 33 job positions, 1,000 employee records, and 1,000 salary records. These tables are connected through primary keys and foreign keys to maintain relationships and data consistency.

OBJECTIVES

The main objectives of the database are:

- To store employee information, including names, contact details, and employment status.
- To organize departments and job positions.
- To record employee salary information.
- To establish relationships between related tables.
- To maintain accurate and organized records.
- To make retrieving and managing employee information easier.

DATABASE OVERVIEW

The employee_management database contains four related tables: departments, positions, employees, and employee_salaries. The database uses mock data with 10 departments, 33 positions, 1,000 employee records, and 1,000 salary records. Also, the total records in the database are 2,043.

DATABASE RELATIONSHIPS

The database uses primary keys and foreign keys to connect related information.

Departments to Employees (Relationship: One-to-Many)
The primary key department_id in departments connects to the foreign key department_id in employees. One department can have many employees.

Positions to Employees (Relationship: One-to-Many)
The primary key position_id in positions connects to the foreign key position_id in employees. One position can be assigned to many employees.

Employees to Employee Salaries (Relationship: One-to-Many**)**
The primary key employee_id in employees connects to the foreign key employee_id in employee_salaries. One employee can have multiple salary records.

These primary and foreign keys link the tables and help maintain data consistency.

DATA DECTIONARY

The data dictionary describes the fields, data types, keys, and purposes of each table in the Employee Management Database.

Departments Table: Stores department information, including the department ID, department name, and location. The department_id serves as the primary key.

Employees Table: Contains employee details such as employee number, name, gender, birth date, email, phone number, hire date, and employment status. The employee_id is the primary key, while department_id and position_id are foreign keys that connect employees to their departments and positions.

Positions Table: Stores job position information, including the position ID, position title, employment level, and base salary. The position_id serves as the primary key.

Employee Salaries Table: Contains salary records, including salary ID, employee ID, salary amount, salary type, effective date, and end date. The salary_id is the primary key, while employee_id is a foreign key linked to the Employees table.

The data dictionary helps explain the structure and purpose of each field, making the database easier to understand, manage, and maintain.

DATABASE NORMALIZATION

Normalization organizes database information into related tables to reduce unnecessary duplication and improve data consistency.

First Normal Form (1NF)

In the first normal form, employee, department, position, and salary information are stored together in one table. Each column contains a single value, and each row represents one employee record. For example, the first record contains the employee's name, contact details, department, position, base salary, and salary amount. However, department and position information may be repeated for multiple employees.

Second Normal Form (2NF)

In the second normal form, the information is separated into employee, department, and salary tables. The employee table stores personal information, the department table stores department details and position information, and the salary table stores salary amounts and effective dates. Primary keys and foreign keys connect the records. This step helps reduce repeated data, although department and position information are still combined in the department table.

Third Normal Form (3NF)

In the third normal form, the data is divided into four tables: employees, departments, positions, and employee_salaries. Department details are stored separately from employee records, and position titles, employment levels, and base salaries are stored in the positions table. Salary amounts and salary dates are stored in the employee salaries table. Foreign keys connect the related records, reducing duplication and improving data consistency.

POSSIBLE QUERY

CRUD Fuctionality

INSERT INTO departments (department_id, department_name, location)

VALUES (‘11’, ‘Analyst’, ‘Main Office’);

UPDATE departments **SET** department_name =’CIT’ **WHERE** department_id = 11;

DELETE FROM departments **WHERE** department_id = '11';

```
SELECT d.department_id, d.department_name, SUM(es.salary_amount) AS Total_salary,
ROUND(AVG(es.salary_amount),2 )AS Average_salary,
MIN(es.salary_amount) AS Minimum_salary
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN employee_salaries es ON e.employee_id = es.employee_id
GROUP BY d.department_name
ORDER BY Average_salary;
```

Output:

employees (10r × 5c)

#	department_id	department_name	Total_salary	Average_salary	Minimum_salary
1	1	Human Resources	3,760,000.0	37,600.0	22,500.0
2	8	Administration	3,843,000.0	38,430.0	22,000.0
3	4	Marketing	3,974,000.0	39,740.0	22,000.0
4	10	Logistics	3,987,000.0	39,870.0	22,000.0
5	6	Operations	4,075,000.0	40,750.0	23,000.0
6	7	Customer Service	4,090,000.0	40,900.0	22,500.0
7	9	Purchasing	4,092,000.0	40,920.0	23,500.0
8	5	Sales	4,125,000.0	41,250.0	23,500.0
9	3	Finance	4,558,000.0	45,580.0	24,500.0
10	2	Information Technology	4,558,000.0	45,580.0	25,000.0

This SQL query summarizes salary information for each department. It uses SUM() to calculate the total salary, AVG() to get the average salary rounded to two decimal places, and MIN() to find the lowest salary. The JOIN statements connect the employee, department, and salary tables, while GROUP BY organizes the results by department. Finally, ORDER BY Average_salary sorts the departments by average salary from lowest to highest.

```
SELECT d.department_name,
   COUNT(DISTINCT e.employee_id) AS total_employees,
   COUNT(DISTINCT p.position_id) AS total_positions,
   COUNT(CASE WHEN e.employment_status = 'Active' THEN 1 END) AS active_employees,
   ROUND(AVG(s.salary_amount), 2) AS average_salary,
   MIN(s.salary_amount) AS lowest_salary,
   MAX(s.salary_amount) AS highest_salary,
   SUM(s.salary_amount) AS total_payroll
FROM departments d
JOIN employees e ON d.department_id = e.department_id
JOIN positions p ON e.position_id = p.position_id
JOIN employee_salaries s ON e.employee_id = s.employee_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.employee_id) >= 50
ORDER BY total_payroll DESC;
```

Output:

departments (10r × 8c)

#	department_name	total_employees	total_positions	active_employees	average_salary	lowest_salary	highest_salary	total_payroll
1	Information Technology	100	5	100	45,580.0	25,000.0	68,000.0	4,558,000.0
2	Finance	100	4	100	45,580.0	24,500.0	66,500.0	4,558,000.0
3	Sales	100	3	80	41,250.0	23,500.0	63,500.0	4,125,000.0
4	Purchasing	100	3	100	40,920.0	23,500.0	61,500.0	4,092,000.0
5	Customer Service	100	3	100	40,900.0	22,500.0	58,500.0	4,090,000.0
6	Operations	100	3	100	40,750.0	23,000.0	64,000.0	4,075,000.0
7	Logistics	100	3	80	39,870.0	22,000.0	61,000.0	3,987,000.0
8	Marketing	100	3	100	39,740.0	22,000.0	61,000.0	3,974,000.0
9	Administration	100	3	100	38,430.0	22,000.0	59,000.0	3,843,000.0
10	Human Resources	100	3	100	37,600.0	22,500.0	59,500.0	3,760,000.0

This SQL query provides a summary of employee and salary information for each department. It counts the total employees, positions, and active employees, then calculates the average, lowest, highest, and total salaries. The JOIN statements connect the department, employee, position, and salary tables. GROUP BY organizes the results by department, while HAVING COUNT(e.employee_id) >= 50 displays only departments with at least 50 employee records. Finally, ORDER BY total_payroll DESC sorts the departments from the highest to the lowest total payroll.

```
SELECT employment_status,
   COUNT(*) AS total_employees
FROM employees
GROUP BY employment_status
ORDER BY total_employees DESC;
```

Output:

employees (3r × 2c)

#	employment_status	total_employees
1	Active	960
2	Inactive	20
3	On Leave	20

This SQL query counts the number of employees in each employment status, such as Active, Inactive, and On Leave. The COUNT(*) function counts the employees in each category, while GROUP BY employment_status groups employees according to their status. Finally, ORDER BY total_employees DESC sorts the results from the employment status with the highest number of employees to the lowest.

```
SELECT e.employee_number,
   CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
   d.department_name, p.position_title, s.salary_amount
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN positions p ON e.position_id = p.position_id
JOIN employee_salaries s ON e.employee_id = s.employee_id
ORDER BY s.salary_amount DESC
LIMIT 10;
```

Output:

employees (10r × 5c)

#	employee_number	employee_name	department_name	position_title	salary_amount
1	EMP-0142	Samantha Navarro	Information Technology	IT Manager	68,000.0
2	EMP-0342	Samantha Navarro	Information Technology	IT Manager	68,000.0
3	EMP-0542	Samantha Navarro	Information Technology	IT Manager	68,000.0
4	EMP-0742	Samantha Navarro	Information Technology	IT Manager	68,000.0
5	EMP-0942	Samantha Navarro	Information Technology	IT Manager	68,000.0
6	EMP-0092	Christine Fernandez	Information Technology	IT Manager	67,000.0
7	EMP-0292	Christine Fernandez	Information Technology	IT Manager	67,000.0
8	EMP-0442	Catherine Navarro	Information Technology	IT Manager	67,000.0
9	EMP-0492	Christine Fernandez	Information Technology	IT Manager	67,000.0
10	EMP-0692	Christine Fernandez	Information Technology	IT Manager	67,000.0

This SQL query displays the **top 10 highest-paid employees** in the company. It uses CONCAT() to combine the employee's first and last names, while the JOIN statements connect the employee, department, position, and salary tables. The ORDER BY s.salary_amount DESC sorts employees from the highest salary to the lowest, and LIMIT 10 displays only the first 10 employees in the results.

```
SELECT MIN(salary_amount) AS lowest_salary,
   MAX(salary_amount) AS highest_salary,
   ROUND(AVG(salary_amount), 2) AS average_salary,
   SUM(salary_amount) AS total_monthly_payroll
FROM employee_salaries;
```

Output:

employee_salaries (1r × 4c)

#	lowest_salary	highest_salary	average_salary	total_monthly_payroll
1	22,000.0	68,000.0	41,062.0	41,062,000.0

This SQL query summarizes the salary information of all employees. The MIN() function finds the lowest salary, while MAX() finds the highest salary. The AVG() function calculates the average salary, rounded to two decimal places using ROUND(). Finally, SUM() calculates the total salary amount from all records in the employee_salaries table. The results provide an overview of the company's salary expenses.

CONCLUSION

In conclusion, the Employee Management Database provides an organized way to store and manage employee information, departments, positions, and salary records. The database uses four related tables to keep the information structured and connected. The data dictionary helps explain the purpose of each table and field, while the Entity Relationship Diagram (ERD) shows how the tables are connected. Through proper normalization up to Third Normal Form (3NF), the database helps reduce data duplication and maintain data consistency. Overall, this database demonstrates how MySQL can be used to manage employee records efficiently and accurately.